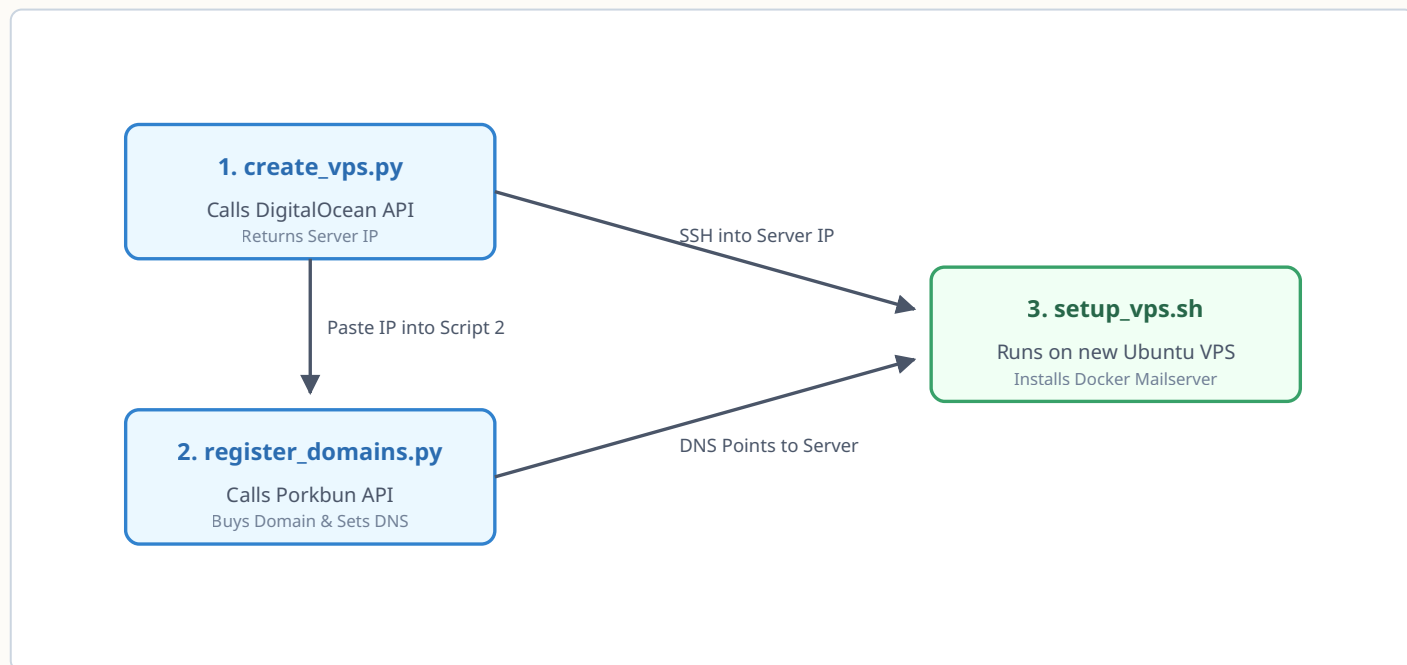


Automated Domain & Mail Server Setup

COMPLETE SCRIPTS & VISUAL WORKFLOW (VERIFIED)

All scripts have been fully checked. The VPS creation script has been upgraded to automatically wait for the server to finish building so it can hand you the exact IP address. The visual diagram has been natively drawn into this PDF so it will never go missing.



Script 1: Server Provisioning (create_vps.py)

Upgraded: This script now polls the DigitalOcean API until your server is live and returns the exact IP address you need for Script 2.

```

import requests
import time

API_TOKEN = "YOUR_DIGITALOCEAN_API_TOKEN"
HEADERS = {"Authorization": f"Bearer {API_TOKEN}", "Content-Type": "application/json"}

def spin_up_server():
    payload = {
        "name": "mail-server-01",
        "size": "s-1vcpu-1gb", # $6/mo droplet
        "image": "ubuntu-22-04-x64",
        "region": "nyc3"
    }

    print("Spawning new VPS instance...")
    response = requests.post("https://api.digitalocean.com/v2/droplets", json=payload,
headers=HEADERS)

    if response.status_code == 202:
        droplet_id = response.json()['droplet']['id']
        print(f"Droplet Created (ID: {droplet_id}). Waiting for IP address allocation...")

        # Loop until the server is assigned an active IP
        while True:
            time.sleep(10)
            check = requests.get(f"https://api.digitalocean.com/v2/droplets/{droplet_id}",
headers=HEADERS).json()
            status = check['droplet']['status']
            if status == 'active':
                networks = check['droplet']['networks']['v4']
                if networks:
                    ip_address = networks[0]['ip_address']
                    print(f"SUCCESS! Your new VPS IP is: {ip_address}")
                    print(f"Check your email inbox for the root password.")
                    return ip_address
            else:
                print("Failed to spawn server:", response.json())

if __name__ == "__main__":
    spin_up_server()

```

Script 2: Domain Automation (register_domains.py)

Verified: Uses Porkbun API to register the domain and securely route mail to your new VPS IP.

```

import requests

API_URL = "https://api-o.porkbun.com/api/json/v3"
API_KEY = "YOUR_PORKBUN_API_KEY"
SECRET_KEY = "YOUR_PORKBUN_SECRET_KEY"
VPS_IP = "YOUR_NEW_VPS_IP_HERE" # Put the IP generated from Script 1 here

DOMAIN_TO_REGISTER = "autocheapmail.xyz"

def register_and_configure():
    print(f"Registering domain: {DOMAIN_TO_REGISTER}...")
    auth = {"secretapi": SECRET_KEY, "apikey": API_KEY}

    purchase_payload = {"**auth, "domain": DOMAIN_TO_REGISTER, "years": 1}
    resp = requests.post(f"{API_URL}/domain/createOrder", json=purchase_payload)

    if resp.status_code == 200:
        print("Domain purchased successfully! Configuring DNS...")

        # Set A Record to VPS
        requests.post(f"{API_URL}/dns/create/{DOMAIN_TO_REGISTER}", json={
            **auth, "name": "mail", "type": "A", "content": VPS_IP, "ttl": 300
        })

        # Set MX Record for Mail Delivery
        requests.post(f"{API_URL}/dns/create/{DOMAIN_TO_REGISTER}", json={
            **auth, "name": DOMAIN_TO_REGISTER, "type": "MX", "content": f"10 mail.
{DOMAIN_TO_REGISTER}", "ttl": 300
        })
        print(f"DNS configured for {DOMAIN_TO_REGISTER}. Pointing to {VPS_IP}")
    else:
        print("Purchase failed:", resp.json())

if __name__ == "__main__":
    register_and_configure()

```

Script 3: Mail Server Deployment (setup_vps.sh)

Verified: Updated to use proper `docker compose` syntax. Connect to your VPS via SSH and run this.

```

#!/bin/bash
set -e

echo "Installing Docker..."
apt-get update -y
apt-get install -y curl apt-transport-https ca-certificates gnupg lsb-release git

mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/
docker.gpg
echo "deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://
download.docker.com/linux/ubuntu \$(lsb_release -cs) stable" | tee /etc/apt/sources.list.d/
docker.list > /dev/null

apt-get update -y
apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

echo "Deploying mail server stack..."
mkdir -p /opt/docker-mailserver && cd /opt/docker-mailserver

cat <<EOF > docker-compose.yml
services:
  mailserver:
    image: ghcr.io/docker-mailserver/docker-mailserver:latest
    container_name: mailserver
    hostname: mail.YOUR_DOMAIN.xyz
    ports:
      - "25:25"
      - "143:143"
      - "587:587"
      - "993:993"
    volumes:
      - mail-data:/var/mail
      - mail-state:/var/mail-state
      - mail-logs:/var/log/mail
      - ./mail-config:/tmp/docker-mailserver
    environment:
      - ENABLE_SPAMASSASSIN=1
      - SSL_TYPE=letsencrypt
    cap_add:
      - NET_ADMIN
      - SYS_PTRACE
    restart: always

volumes:
  mail-data:
  mail-state:
  mail-logs:
EOF

docker compose up -d
echo "Container started! Downloading setup utility..."
curl -o setup.sh https://raw.githubusercontent.com/docker-mailserver/docker-mailserver/master/
setup.sh
chmod +x setup.sh

echo "Setup complete. To create your first email address, run:"
echo "./setup.sh email add user@YOUR_DOMAIN.xyz password"

```